

```
print("Hello, World!")
```



LINGUAGEM DE PROGRAMAÇÃO PYTHON



AULA 04

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Apresentação de strings;**
- **Concatenação;**
- **Repetição;**
- **f-strings;**
- **Métodos para tratamentos de strings;**

PRÓXIMOS PASSOS

- **Condições lógicas;**
- **Onde utilizar estruturas condicionais;**
- **Estrutura condicional simples (if);**
- **Estrutura condicional composta (elif e else);**
- **match...case**
- **Apresentação dos tipos de laços de repetição;**
- **Estrutura de looping (While);**

O que são estruturas condicionais

- Estruturas que permitem executar um bloco de código apenas se uma condição for atendida
- Para verificar se a condição foi atendida seu valor deve ser verdadeiro (**True**) ou falso (**False**)
- Decisão: Se a condição for **True**, o código entra no bloco e executa a instrução; se for **False**, o código simplesmente ignora esse bloco e segue em frente.

Exemplo de uso de estruturas condicionais

Em uma tela de login o usuário insere seu login e senha.

Caso esses login e senha sejam válidos (True), o login é realizado.

Caso o login e senha estejam errados (False), o login não é realizado.

Mas como verificar se uma condição é verdadeira?

Operadores relacionais são utilizados para comparar dois valores e retornar **True** ou **False** na comparação

== (Operador igual)

!= (Operador diferente)

> (Operador maior)

< (Operador menor)

>= (Operador maior ou igual)

<= (Operador menor ou igual)

Variáveis com tipos diferentes

- **Valores aparentes enganam:** Para nós, "7" e 7 representam a mesma coisa, mas para o computador a string "7" é um caractere (como a letra 'A') e o número 7 é um valor matemático.
- Solução: Para comparar esses valores, você precisa converter os tipos primeiro usando `int()` ou `str()`:

Tipos de variáveis

```
# O professor Gandalf registra a nota do aluno Bilbo:
```

```
nota_texto = "7"      # Tipo: str (String / Texto)
```

```
nota_numero = 7       # Tipo: int (Inteiro / Número)
```

```
# Comparando se a nota em texto é igual à nota em número:
```

```
resultado = (nota_texto == nota_numero)
```

```
|
```

```
print("Nota em texto:", type(nota_texto))
```

```
print("Nota em número:", type(nota_numero))
```

```
print("As duas variáveis são iguais?", resultado)
```

```
Nota em texto: <class 'str'>
```

```
Nota em número: <class 'int'>
```

```
As duas variáveis são iguais? False
```


Atividade Prática 01 Aula 04

- Imagine que você precisa comparar as pontuações de dois jogadores ao final de uma partida.
- Crie duas variáveis atribuindo valores numéricos a cada uma delas.
- Exiba na tela o resultado de todas as seis operações relacionais estudadas, comparando a pontuação do jogador 1 com a do jogador 2.
- Você deve escrever uma frase explicativa antes de cada resultado para identificar o que está sendo comparado.
- Atenção: Não utilize estruturas condicionais (como if ou else). O objetivo é apenas observar a saída de True ou False diretamente no console.

Operadores lógicos

São utilizados para definir condições mais complexas e que dependem de mais de uma comparação ou de uma negação.

- **and** - Operador **E** (retorna verdadeiro quando as condições que ele está combinando são verdadeiras)
- **or** - Operador **OU** (retorna verdadeiro quando qualquer uma das condições que ele está combinando são verdadeiras)
- **not** - Operador **negação** (retorna o resultado oposto da comparação que ele está modificando)

Podem ser combinados múltiplos operadores lógicos para montar uma comparação mais complexa.

Operador and

and - Operador **E** (retorna verdadeiro quando as condições que ele está combinando são verdadeiras)

Condição A	Condição B	Resultado (A and B)
FALSO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
VERDADEIRO	FALSO	FALSO
VERDADEIRO	VERDADEIRO	VERDADEIRO

Operador and

```
# Ambas as condições precisam ser True
```

```
tem_convite = True
```

```
esta_na_lista = True
```

```
# O acesso só é liberado se tiver convite E estiver na lista
```

```
acesso_liberado = tem_convite and esta_na_lista
```

```
print("Acesso liberado?", acesso_liberado)
```

```
# Saída: True
```

Acesso liberado? True

Operador or

or - Operador **OU** (retorna verdadeiro quando qualquer uma das condições que ele está combinando são verdadeiras).

Condição A	Condição B	Resultado (A or B)
FALSO	FALSO	FALSO
FALSO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
VERDADEIRO	VERDADEIRO	VERDADEIRO

Operador or

```
tem_anel = False  
e_o_gandalf = True
```

```
# Apenas UMA condição precisa ser True  
acesso_liberado = tem_anel or e_o_gandalf
```

```
print("Acesso liberado?", acesso_liberado)  
# Saída: True
```

Acesso liberado? True

Operador not

not - Operador **negação** (retorna o resultado oposto da comparação que ele está modificando)

Condição A	Resultado (not A)
FALSO	VERDADEIRO
VERDADEIRO	FALSO

Operador not



```
A = True
```

```
B = False
```

```
print (not A)
```

```
print (not B)
```

```
... False  
True
```

Operador xor

OU Exclusivo (conhecido na lógica e na programação como **XOR**): ele só é verdadeiro quando uma das condições é verdadeira, mas **NÃO** as duas ao mesmo tempo.

Condição A	Condição B	Resultado ($A \neq B$ ou $A \wedge B$)
FALSO	FALSO	FALSO
FALSO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
VERDADEIRO	VERDADEIRO	FALSO

Operador xor

```
A = True
```

```
B = False
```

```
print (A != A)
```

```
print (A != B)
```

```
print (B != A)
```

```
print (B != B)
```

False

True

True

False

Atividade Prática 02 Aula 04

Você está criando o sistema de segurança de um aplicativo. Para que o usuário consiga fazer o login com sucesso, algumas regras precisam ser atendidas simultaneamente:

- O nome de usuário digitado deve ser igual ao cadastrado no sistema.
- A senha digitada deve ser igual à cadastrada.
- A conta não pode estar bloqueada.

Crie as variáveis necessárias e escreva expressões lógicas que retornem True ou False para as seguintes verificações, imprimindo os resultados na tela:

- As credenciais (usuário e senha) estão corretas?
- A conta está bloqueada?
- O acesso final está liberado (credenciais corretas E conta NÃO bloqueada)?

Testando uma comparação

Para testar uma comparação e executar um bloco de códigos com base no resultado usamos o **if**.

Se o resultado for verdadeiro o código será executado, caso seja falso ele é ignorado.

No Python o bloco de código abrangido pela condição é determinado pela indentação (espaço a esquerda).

if

```
senha_digitada = input("Digite a senha, amigo: ")

# Comparando se o texto digitado é IGUAL ao esperado
if senha_digitada == "mellon":
    print("A porta se abre! Bem-vindo às Minas de Moria.")
```

E se eu quiser executar um comando em caso da comparação ser falsa?

Podemos utilizar os operadores else e elif:

- **elif** é utilizado para se testar uma nova condição caso a condição **if** anterior seja falsa
- **else** é utilizado para executar um código padrão caso todas as condições anteriores (**if e elif**) sejam falsas.

if e else

```
senha_digitada = input("Digite a senha, amigo: ")

# Comparando se o texto digitado é IGUAL ao esperado
if senha_digitada == "mellon":
    print("A porta se abre! Bem-vindo às Minas de Moria.")
else:
    print("Senha incorreta! A porta continua trancada.")
```

```
Digite a senha, amigo: amigo
Senha incorreta! A porta continua trancada.
```


if, elif e else

```
senha_digitada = input("Digite a senha, amigo: ")

# Comparando se o texto digitado é IGUAL ao esperado
if senha_digitada == "mellon":
    print("A porta se abre! Bem-vindo às Minas de Moria.")
elif senha_digitada == "amigo":
    print("Não é bem isso")
else:
    print("Senha incorreta! A porta continua trancada.")
```

```
Digite a senha, amigo: amigo
Não é bem isso
```

Atividade Prática 03 - Notas

Escreva um programa em Python que:

- Pergunte ao usuário qual foi a sua nota (lembre-se de converter para float, pois as notas podem ter decimais).
- Utilize as estruturas condicionais if, elif e else para avaliar a nota e exibir na tela o título conquistado pelo aluno, para cada uma das condições abaixo deve-se exibir uma mensagem diferente:
- Nota 9.0 ou mais;
- Nota de 7.0 até 8.9;
- Nota de 5.0 até 6.9;
- Nota abaixo de 5.0;

Atividade Prática 04 - A Jornada do Personagem/Usuário (Operadores e Condicionais)

- Agora que você já sabe capturar dados com `input()`, converter tipos e fazer cálculos, chegou a hora de dar inteligência e consequências para a história do seu personagem/usuário usando o comando `if`!
- Anteriormente, usar operadores relacionais (`>`, `<`, `==`) apenas mostrava `True` ou `False` na tela.
- Agora, você vai usar essas comparações para fazer o computador tomar decisões.

Atividade Prática 04 - A Jornada do Personagem/Usuário (Operadores e Condicionais)

- **A Missão:** Crie (ou continue) um pequeno script onde o seu personagem/usuario toma ações que envolvem cálculos e decisões.
- **Cálculos e Variáveis:** Crie pelo menos 3 novas variáveis que envolvam operações aritméticas (ex: calcular o dano de um ataque, subtrair dinheiro após uma compra, somar pontos de experiência).
- **Tomada de Decisão (if / else):** Baseado nos resultados desses cálculos, crie 3 testes condicionais para verificar o estado do seu personagem.
- **Mostre o que aconteceu na história usando o print().**

Próximos Passos



match...case

Apresentação dos tipos de laços de repetição;

Estrutura de looping (While);

**Estrutura condicional composta (For) e função
range();**

match...case

- Utilizado para verificar se uma variável é igual a uma série de valores (cases);
- Caso seja igual a um case, executa o bloco de código correspondente;
- “_” pode ser utilizado como default para executar um código caso nenhum outro case seja o correto.

match...case (exemplo)

```
personagem = input("Digite o nome do personagem: ").lower().strip()

match personagem:
    case "frodo":
        print("Carrega o Um Anel.")
    case "legolas":
        print("Mestre dos arcos e flechas.")
    case "gimli":
        print("Guerreiro anão com seu machado.")
    case _:
        # O "_" funciona como o "default" (caso nenhum outro bata)
        print("Membro desconhecido da comitiva.")
```

```
Digite o nome do personagem: FRODO
Carrega o Um Anel.
```


Atividade Prática 05 Aula 04

Sistema de Chamados (Help Desk)

Você foi contratado para desenvolver a triagem de um sistema de chamados de TI. O programa deve pedir ao usuário para digitar o número correspondente ao problema que ele está enfrentando. Utilizando a estrutura match...case, direcione o chamado para a equipe correta e exiba uma mensagem confirmando o direcionamento, o menu de escolhas deve ter pelo menos 4 opções.

Exemplos de Opções:

- 1 - Problema com Computador/Hardware → Direcionar para: Manutenção
- 2 - Instalação de Software → Direcionar para: Equipe de Suporte
- 3 - Sem acesso à Internet/Rede → Direcionar para: Infraestrutura
- 4 - Reset de Senha → Direcionar para: Atendimento Automático (Envio de link de redefinição)

Qualquer outra opção → Exibir: Opção inválida. Chamado encaminhado para a Triagem Geral.

O que são estruturas de repetição?

- São blocos de código que permitem a execução repetitiva de uma instrução ou conjunto de instruções, desde que uma determinada condição seja satisfeita;
- Existem 2 estruturas de repetição no Python:
 - while;
 - for.

Por que usar estruturas de repetição?

- Evita repetir o mesmo código várias vezes manualmente;
- Nem sempre sabemos previamente a quantidade de vezes que um comando deve ser executado para completar o objetivo;
- São muito úteis para automatizar processos repetitivos e processar (percorrer) grandes quantidades de dados.

While (enquanto)

- O loop while em Python é uma estrutura de repetição que executa um bloco de código enquanto uma condição for verdadeira;
- Ele é mais útil quando não sabemos previamente a quantidade de vezes que um comando será repetido;
- Cuidado com loops infinitos!
- Sua sintaxe é a seguinte:

while condicao:

 #comandos a serem repetidos

Incrementos e Decrementos

O que são?

São operações usadas para atualizar o valor de uma variável, geralmente adicionando ou subtraindo uma quantidade fixa a cada repetição de um loop.

- Incremento (Soma): Faz a variável crescer. Essencial para contar passos ou somar pontuações.
- Decremento (Subtração): Faz a variável diminuir. Essencial para contagens regressivas ou simular o consumo de algo.

O Atalho: Em Python, em vez de escrever a variável duas vezes (`variavel = variavel + 1`), usamos os operadores **`+=`** e **`-=`**.

While (Exemplo)

```
distancia_mordor = 3  # distância restante

# O bloco repete ENQUANTO a condição for True
while distancia_mordor > 0:
    print(f"Frodo andou. Faltam {distancia_mordor} km.")
    distancia_mordor -= 1  # Decrementa para evitar loop infinito

print("Frodo chegou à Montanha da Perdição!")
```

Frodo andou. Faltam 3 km.

Frodo andou. Faltam 2 km.

Frodo andou. Faltam 1 km.

Frodo chegou à Montanha da Perdição!

for (para)

- O loop **for** é usado para percorrer elementos de uma sequência, como listas, tuplas, strings, dicionários e até intervalos numéricos;
- Ideal quando já sabemos quantas vezes queremos repetir o código ou queremos iterar sobre uma coleção de dados;
- Sua sintaxe é a seguinte:

for variavel in sequencia:

#codigo que se repete para manipular variavel

for (Exemplo)

```
palavra = "Python"
```

```
for letra in palavra:  
    print(letra)
```

P
y
t
h
o
n

for (Exemplo)

```
comitiva = ["Frodo", "Sam", "Aragorn", "Legolas"]
```

```
# Percorre cada elemento da lista
```

```
for membro in comitiva:
```

```
    print(f"Membro da comitiva: {membro}")
```

```
Membro da comitiva: Frodo
```

```
Membro da comitiva: Sam
```

```
Membro da comitiva: Aragorn
```

```
Membro da comitiva: Legolas
```


A Função **range()** (O Gerador de Números)

O que é? Uma função do Python que cria automaticamente uma sequência de números.

Para que serve? Quando queremos repetir uma ação um número exato de vezes, sem precisar criar uma lista manualmente.

A Regra: O range sempre começa do zero (por padrão) e para um número antes do valor final.

Exemplos :

- **range(5)** → Gera: 0, 1, 2, 3, 4 (Repete 5 vezes)
- **range(1, 6)** → Gera: 1, 2, 3, 4, 5 (Começa no 1, para antes do 6)
- **range(0, 10, 2)** → Gera: 0, 2, 4, 6, 8 (O terceiro número é o "passo", pula de 2 em 2)

range (Exemplo)

```
for dia in range(1, 4):  
    print(f"Dia {dia}: A comitiva continua marchando...")  
  
print("Chegaram ao destino final!")
```

```
Dia 1: A comitiva continua marchando...  
Dia 2: A comitiva continua marchando...  
Dia 3: A comitiva continua marchando...  
Chegaram ao destino final!
```

Atividade Prática 06 Aula 04

Sistema de Caixa e Parcelamento

Objetivo: Desenvolver um programa que utilize os dois tipos de laço de repetição (while e for).

Cenário: Você está programando o sistema de caixa de uma loja. O programa deve funcionar em duas etapas:

Passando as compras: O sistema deve pedir ao caixa para digitar o preço de cada produto. Como não sabemos quantos produtos o cliente vai levar, o programa deve continuar pedindo os preços e somando o total até que o caixa digite o valor 0 (zero), que significa que a compra acabou.

Opções de Parcelamento: Após calcular o total, o sistema deve mostrar ao cliente as opções de parcelamento, mostrando o valor de cada parcela.

Ao final do código, adicione um comentário (#) respondendo: Qual laço de repetição foi a melhor escolha para a etapa 1 e 2 e porque?

Próximos Passos



**Continue e break;
Apresentação de funções;
Exemplo de utilização;
Estrutura de uma função em Python;**

**exit() ou
dúvidas?**

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

OBRIGADA!

“Sparse is better than dense.”
The Zen of Python, by Tim Peters



SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais
informações
acesso
o **QR CODE**
e saiba mais



 (41)3052-4900

 @unisantacruz

 /UniSantaCruzCtba

 unisantacruztem.com.br